

CSCE 620 COMPUTATIONAL GEOMETRY · MODULE 04 · LECTURE NOTES

Plane Sweep and Segment Intersection

One idea that solves a dozen problems.

MODULE OUTCOMES

Explain the plane-sweep paradigm and its two structures.

Implement Bentley–Ottmann segment intersection.

Analyse an output-sensitive sweep.

Handle the sweep degeneracies by name.

Recognise other problems the sweep solves.

1 The paradigm

SWEEP REDUCES A 2D PROBLEM TO A 1D PROBLEM THAT CHANGES

Imagine a vertical line sweeping continuously from left to right across the plane. At any instant it meets some subset of the input, and that subset has a natural one-dimensional order along the line.

Maintain that subset in sorted order — the *status structure*, normally a balanced binary search tree supporting insert, delete, swap, and neighbour queries in logarithmic time.

The order changes only at discrete *events*, held in a priority queue ordered by sweep position: segment left endpoints, right endpoints, and — discovered during the sweep — intersections.

So the continuous sweep is simulated by jumping from event to event. Nothing whatsoever is examined between events, and that is where every bit of the saving comes from. The geometry is reduced to maintaining an order.

INTERSECTIONS HAPPEN BETWEEN NEIGHBOURS

Two segments can only intersect if they become adjacent in the status order at some moment before they cross. Immediately before the crossing they are neighbours — nothing can lie between them — so the adjacency must have arisen at some earlier event.

So you never test all pairs. You test only pairs that actually become neighbours, and each event changes only a constant number of adjacencies, generating a constant number of new tests.

That reduces $O(n^2)$ candidate pairs to $O(n + k)$ actual tests, where k is the number of intersections reported.

The proof of that claim is essentially the whole algorithm. Everything else — the three event handlers, the tie-breaking, the queue discipline — is bookkeeping in service of keeping the status order correct so that the adjacency argument remains valid.

```

EVENT QUEUE Q: endpoints ordered by (x, then y)
STATUS      T: segments crossing the sweep line, ordered by y

while Q not empty:
    p = Q.pop()

    LEFT ENDPOINT of s:
        insert s into T
        test s against its new neighbours above and below
        queue any intersection strictly right of p

    RIGHT ENDPOINT of s:
        remove s from T
        its two former neighbours are now adjacent: test THAT pair

    INTERSECTION of s and t:
        report it; swap s and t in T
        test each against its new OUTER neighbour

 $O((n + k) \log n)$ 

```

All three handlers do the same two things: update the status order, then test exactly the adjacencies that the update created. Holding the algorithm in that form makes it far easier to reconstruct than memorising three separate cases.

2 Cost

$$O((n + k) \log n)$$

for n segments and k reported intersections. Each event costs one logarithmic operation on the priority queue and one on the status tree, and there are $n + k$ events. **This beats $O(n^2)$ whenever k is small**, which it usually is in practice — map data, floor plans, and polygon overlays all have far fewer crossings than the quadratic worst case. **But when k approaches n^2 , brute force wins:** it has no logarithmic factor, no tree, and perfect memory locality. **Chazelle and Edelsbrunner achieved the optimal $O(n \log n + k)$,** which

separates the two terms, at a considerable cost in implementation complexity that is rarely repaid.

3 Degeneracy in the sweep

Degenerate case	What it breaks	How to handle it
A vertical segment.	It has no single x position, so its place in the status order is undefined and it generates two events at the same x.	Conceptually tilt the sweep line infinitesimally (equivalently, order events by x then y), or special-case verticals explicitly. The tilt is cleaner and is symbolic perturbation (Module 02) again.
Two segments sharing an endpoint.	Two events occur at exactly the same point.	Process all events at a given point together as a single composite event, rather than one at a time.
Three or more segments through one point.	The pairwise swap is wrong — see the callout below.	Reverse the entire bundle at once , and test only the two outermost members against their new neighbours.
Overlapping collinear segments.	The intersection is a segment, not a point, so the output type is wrong.	Decide what the output means for this case and define it explicitly. There is no universally right answer.
An endpoint lying on another segment's interior.	Is this an intersection or not?	A specification question, not a geometry question. Both answers are defensible; the caller must be told which it gets.
Several events at equal x.	The event order is ambiguous, and the wrong order corrupts the status.	Break ties by y, then by event type — with a fixed, documented precedence among left endpoint, intersection, and right endpoint.

WHY THREE-THROUGH-ONE-POINT IS THE GENUINELY HARD CASE

At an ordinary crossing of two segments, their relative order in the status structure reverses, and a single swap handles it correctly.

When m segments pass through a common point, the order of all m reverses — the whole bundle flips as a block, so the segment that was topmost becomes bottommost.

Processing that as a sequence of pairwise swaps produces the wrong final order, because the intermediate states are not valid configurations of the sweep. The consequences are reported intersections that do not exist, missed intersections that do, and in some implementations a corrupted tree.

So collect every event at the same point, reverse the bundle in one operation, and then test only the two outer boundaries of the bundle against their new neighbours. Note that this configuration has probability zero under random input (Module 01 §4) and is entirely routine in real data — any grid, any symmetric model, any CAD drawing. It is the single best illustration in this course of why random testing proves nothing.

4 The same idea elsewhere

- **Closest pair of points** — sweep while maintaining a strip of candidates within the current best distance. $O(n \log n)$.
- **Rectangle union area, and the Klee measure problem** — sweep in x with a segment tree over y tracking covered length.
- **Trapezoidal decomposition**, which underlies planar point location (Module 07).
- **Voronoi diagrams** — Fortune's algorithm sweeps a parabolic beach line rather than a straight status line (Module 08), which is the most inventive use of the paradigm here.
- **Polygon triangulation** by decomposition into monotone pieces (Module 05), where the sweep classifies vertices and adds diagonals.
- **Boolean operations on polygons** (Module 06) and **visibility computations** (Module 11), both of which are sweeps with different status contents.

Six of the remaining modules in this course are plane sweeps. The paradigm is worth learning once, thoroughly, because the investment is repaid repeatedly — and because recognising a problem as a sweep is most of the work of solving it.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
de Berg et al. — Computational Geometry, chapter 2 (line segment intersection)	The canonical treatment of Bentley–Ottmann, including the degeneracy handling of §3.
Mount — CMSC 754, plane sweep lectures (free PDF)	The paradigm stated generally, then applied, which is the right order for it.
Bentley & Ottmann — Algorithms for reporting and counting geometric intersections	The original. Short, and worth reading for how directly the adjacency argument is made.
CGAL — 2D Arrangements package documentation (free)	A production implementation that handles every degeneracy in §3's table, and documents how.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement brute-force segment intersection as a reference implementation.
- 2 Implement Bentley–Ottmann and verify it against the reference on random input.

- 3 **Then verify it on degenerate input** — and expect the reference to need fixing too.
- 4 Construct each of the six cases in §3's table explicitly and confirm your handling.
- 5 **Build an input with ten segments through one point** and demonstrate that pairwise swapping gives the wrong order.
- 6 Measure the crossover between brute force and the sweep as k varies from 0 to n^2 .
- 7 Plot the status structure size over the sweep for several inputs and explain the shapes.
- 8 Implement closest pair by sweep and compare against divide and conquer.
- 9 Implement rectangle union area by sweep.
- 10 Draw every intermediate sweep state for a small input as SVG and check them by eye.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Describe the sweep paradigm and name its two data structures.
- 2 Why can intersections only occur between status neighbours?
- 3 Give the three event types and say what each does.
- 4 State the Bentley–Ottmann bound and say when brute force wins.
- 5 Name six sweep degeneracies and their handling.
- 6 Why does a pairwise swap fail when three segments meet at a point?
- 7 Why does random input never produce that case?
- 8 Name six other problems solved by sweeping.